On an adaptive multi-fidelity Gaussian process metamodel using a collective learning strategy[1]

T. Goumont, I. Ben Moussa, L. Gruchociak

École Nationale de la Statistique et de l'Analyse de l'Information (ENSAI)

Collaborative work with the Direction Générale de l'Armement (DGA)

[1] C. Zhang, C. Song, and A. Shafieezadeh, *Adaptive Reliability Analysis for Multi-fidelity Models using a Collective Learning Strategy*, in "Structural Safety", vol.94, 2022.

Overview

1. Introduction
   - What is reliability analysis ?
   - Introducing active learning methods
   - Meta-modeling and multi-fidelity approaches
2. The AMGPRA method explained
   - Surrogate-modeling using Gaussian process regression
   - The proposed procedure
3. Application: AMGPRA in practice
4. Case study: solar
   - Problem physics and modeling
   - From optimization to reliability
   - Next steps
5. Conclusion

Introduction

The reliability analysis framework

Goal: quantify the probability that a system fails [1], *i.e.* that the **limit-state function** $g : \mathcal{X} \to \mathbb{R}$ becomes negative:

$$P_f = \mathbb{P}\big(g(\mathbf{x}) \leq 0\big),\ \mathbf{x} \in \mathcal{X} = \mathbb{R}^d$$

Context: in engineering, $g(\mathbf{x})$ is often

- computationally expensive
- dependent on several uncertain inputs

Challenge: Estimating P_f accurately while limiting the number of expensive simulations

Why not direct Monte Carlo Simulation?

Monte Carlo Simulation (MCS)

- Produces an estimate $\hat{P}_f$ by sampling many realizations of $\mathbf{x} \in \mathcal{X}$
- Simple, robust and asymptotically exact (Law of large numbers)

However:

- Each evaluation of $g(\mathbf{x})$ is **computationally expensive**
- Rare events ($P_f \ll 1$) require a lot of evaluations, up to 10^5–10^6

Therefore:

- Direct MCS is often infeasible in engineering applications
- **Smarter sampling strategies** are needed

Active learning: general idea

Key question in reliability analysis:

In $\mathcal{X}$, where should we evaluate the model next ?

Active learning principle:

- Because the model is costly, do not evaluate the model everywhere on $\mathcal{X}$
- Focus evaluations where they are **the most informative**

In reliability problems,

- Most useful points are close to the **limit-state boundary** *i.e.* $g(\mathbf{x}) = 0,\ \mathbf{x} \in \mathcal{X}$
- They are especially located where model uncertainty is high

The use of surrogate modeling

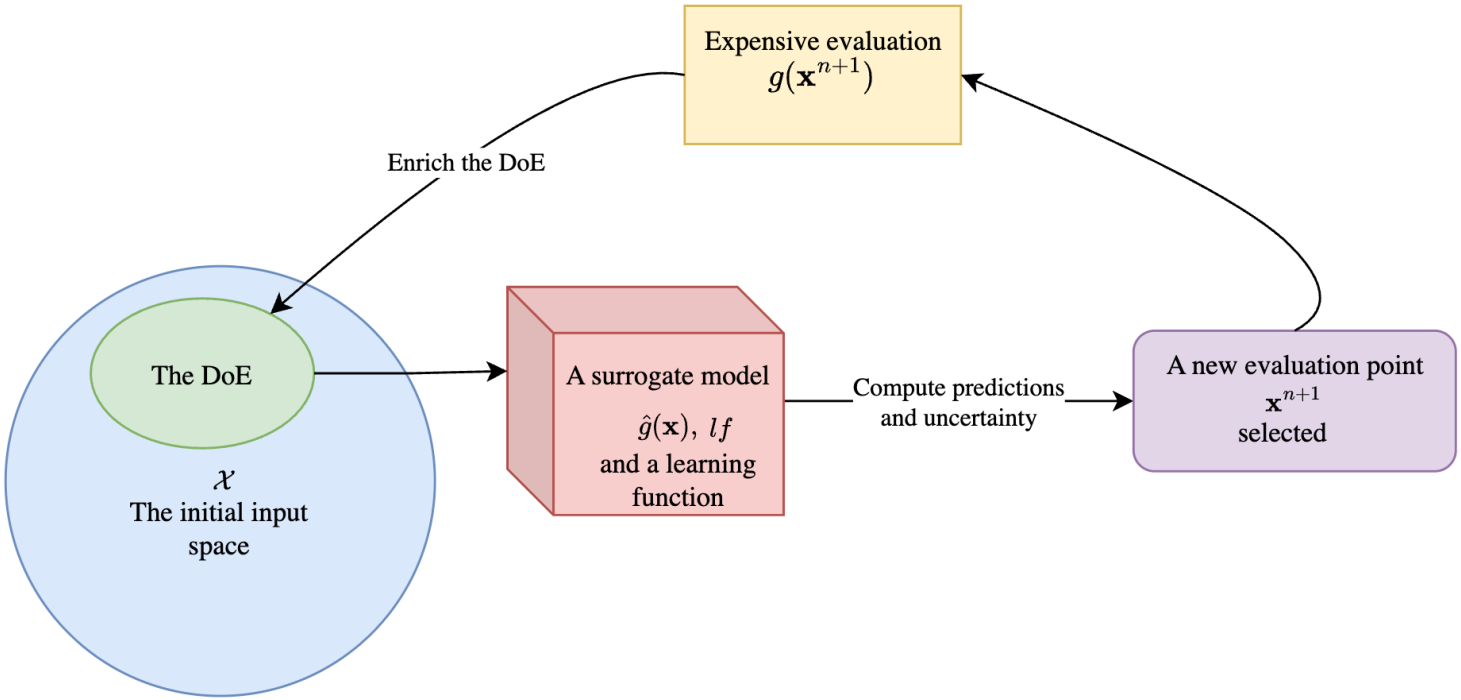

Figure: An active learning procedure using a surrogate model

Example of metamodel: *Gaussian process regression, or Kriging*

From single- to multi-fidelity reliability methods

Different approaches: using high- or low-fidelity sources of information

Single-fidelity → **EGRA** (2008) [2], **AK-MCS** (2011) [3]

- Adaptive Kriging-based surrogates, with different learning functions
- **Limitation:** all new points require expensive high-fidelity evaluations

Multi-fidelity → **mfEGRA** (2020) [4]

- Gaussian process surrogate
- Two-step strategy: select the input point, then the fidelity level
- **Limitations:** can lead to suboptimal selection, expensive

Motivation: jointly selecting point and fidelity → **AMGPRA** (2022) [5]

The AMGPRA method explained

Gaussian process–based metamodels

Gaussian process (GP) regression - Kriging

- A GP defines a probability distribution over functions:

$$\forall\, \mathbf{x}, \mathbf{x}' \in \mathcal{X},\ f(\mathbf{x}) \sim \mathcal{GP}\big(\mu(\mathbf{x}),\ k(\mathbf{x}, \mathbf{x}')\big)$$

- GP-based surrogates rely on this method

Key components

- Mean function: $\mu(\mathbf{x}) = \mathbb{E}[f(\mathbf{x})]$
- Covariance kernel: $k(\mathbf{x}, \mathbf{x}') = \mathrm{Cov}\big(f(\mathbf{x}), f(\mathbf{x}')\big)$

Why GP surrogates for reliability analysis?

- Provide both a prediction and an uncertainty estimate
- Predictive variance $\hat{\sigma}^2_{GP}(\mathbf{x})$ guides active learning
- Kernel choice (*e.g.*, Gaussian, Matérn) controls smoothness and correlation length

An illustration of Gaussian process Regression

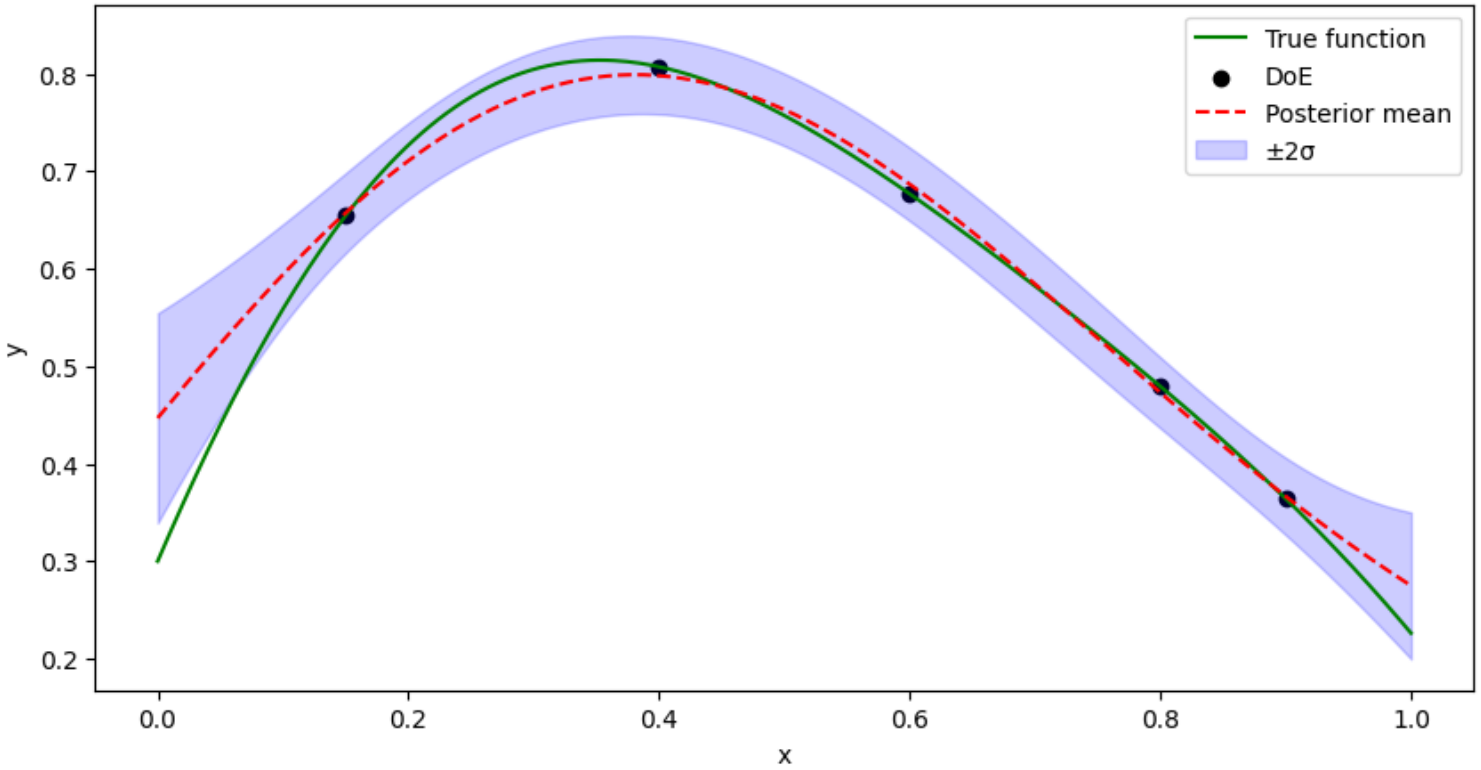

Figure: GP regression on $f(x) = \sin(3x) + 0.5\cos(5x)$ using 5 training points.

AMGPRA framework

Model Definitions

- $K+1$ models of the limit-state function:
  - $l = 0 \rightarrow$ high-fidelity model
  - $l = 1, \ldots, K \rightarrow$ low-fidelity models
- High-fidelity GP:

$$\hat{g}(0, \mathbf{x}) \sim \mathcal{GP}(\mu_0, \Sigma_0), \ \mathbf{x} \in \mathcal{X}$$

- Low-fidelity models are defined through their discrepancy and are also GPs:

$$\delta_l(\mathbf{x}) = \hat{g}(l, \mathbf{x}) - \hat{g}(0, \mathbf{x}) \sim \mathcal{GP}(\mu_l, \Sigma_l), \ \mathbf{x} \in \mathcal{X}$$

Assumptions and estimation:

- Mean functions constant, covariance kernels Gaussian, no noise
- The estimated GP is: $G_P(l, \mathbf{x}^*)$

A new learning function

The collective learning function (CLF):

- Select simultaneously:
  - the next training point $\mathbf{x}^{n+1}$ to evaluate,
  - the information source I_F

Principle

- CLF evaluates the *global impact* of adding $(\mathbf{x}^{n+1}, I_F)$ to the DoE
- measures the average improvement of the learning function over all points in the population, normalized by the cost, *i.e.*

$$CLF(\mathbf{x}^{n+1}, I_F) = \frac{\sum_{i=1}^{N_c} \left[If\big(\mu_P(0,\mathbf{x}^{(i)}),\, \sigma_P^2(0,\mathbf{x}^{(i)})\big) - If\big(\mu_P(0,\mathbf{x}^{(i)}),\, \sigma_F^2(0,\mathbf{x}^{(i)} \mid \mathbf{x}^{n+1}, I_F)\big) \right]}{N_c\, c_I(\mathbf{x}^{n+1})}$$

- Select the $(\mathbf{x}^{n+1}, I_F)$ maximizing the CLF

The AMGPRA procedure

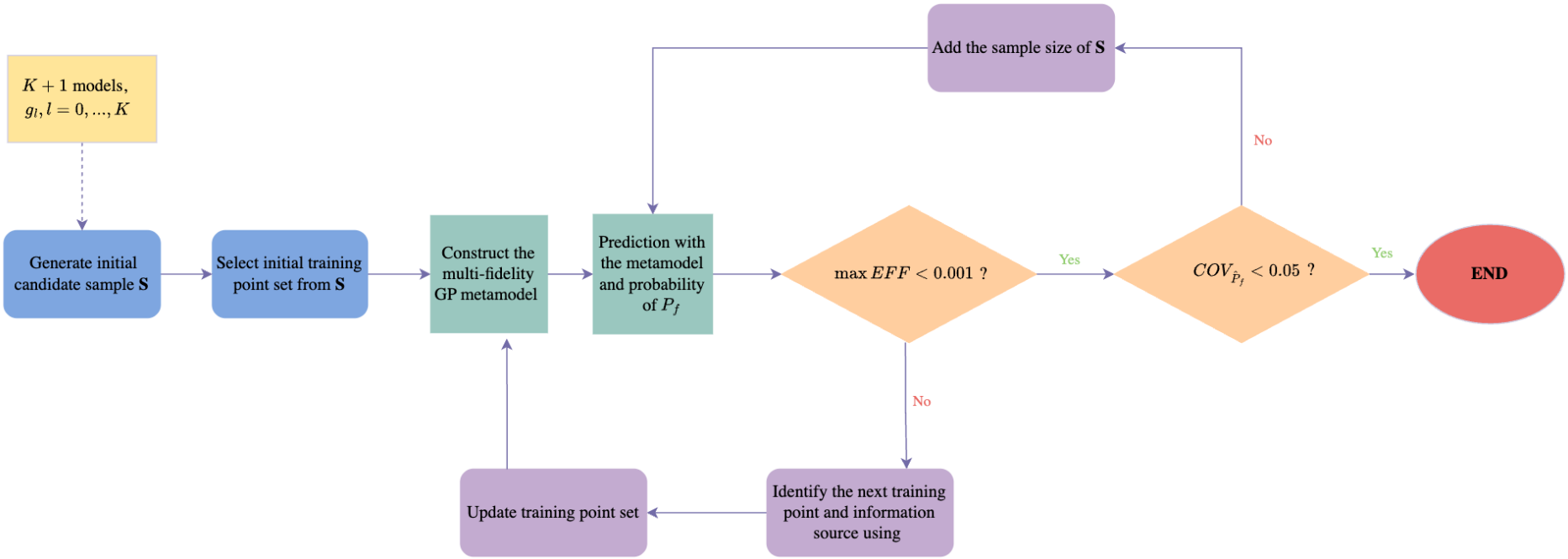

Figure: Flowchart of the AMGPRA approach

Application and reproducibility: AMGPRA in practice

A tri-fidelity example

Problem setting [2]

- Input space: $\mathbf{x} = (x_1, x_2) \sim \mathcal{N}_2(\begin{pmatrix} 1.5 \\ 2.5 \end{pmatrix}, \begin{pmatrix} 1 \\ 1 \end{pmatrix})$
- High-fidelity limit-state function:

$$g_0(x_1, x_2) = \frac{(x_1^2 + 4)(x_2^2 - 1)}{20} - \sin\left(\frac{5x_1}{2}\right) - 2$$

- Two low-fidelity models defined as biased approximations:

$$g_l(\mathbf{x}) = g_0(\mathbf{x}) + \delta_l(\mathbf{x}), \qquad l = 1, 2$$

A tri-fidelity example

Results (reference: Monte Carlo) - Source: [5]

Method	Cost	$\hat{P}_f$	ARE (%)
MCS (reference)	10^6	0.0313	-
AK-MCS	45.2	0.0313	0.04
mfEGRA	12.87	0.0311	0.02
AMGPRA	**12.3**	**0.0310**	**0.02**

Conclusion: AMGPRA achieves similar accuracy with the lowest computational cost.

A tri-fidelity example

- The multi-fidelity metamodel provides an accurate approximation of the limit-state function in the critical regions

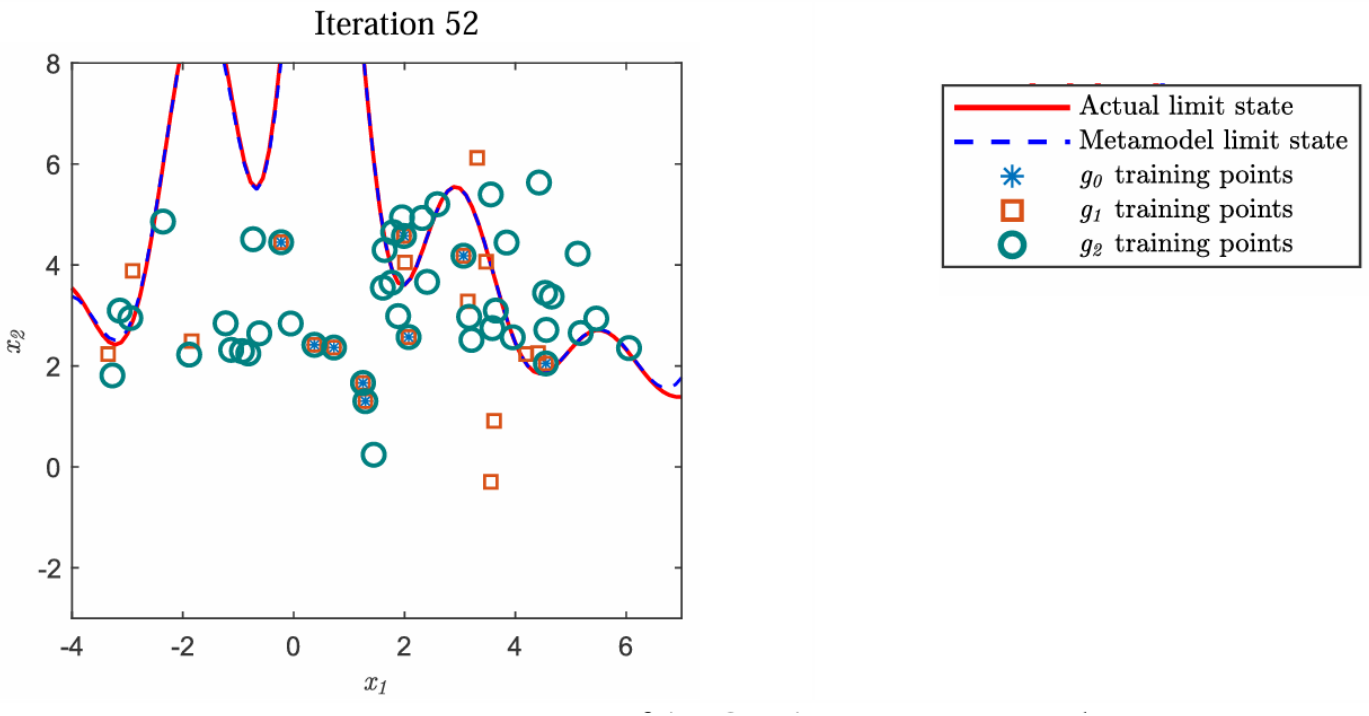

Figure: Evolution of the Design of Experiments - Source: [5]

Case study: The **solar** optimization benchmark[a]

[a] Audet et al., *solar: A solar thermal power plant simulator for blackbox optimization benchmarking, 2024.*

solar: A power plant simulator

solar: benchmark that simulates a concentrated solar power (CSP) plant

- **Heliostats:** Sun-tracking mirrors
- **Receiver:** Captures solar flux to heat molten salt
- **Storage:** Thermal tanks for deferred energy use

Figure: The Thémis CSP tower plant in Targasonne, France

→ High-dimensional blackbox where each evaluation involves expensive ray-tracing and thermodynamic simulations

The Solar 10.1 instance

Cost Function

solar 10.1 instance aims to minimize the storage cost via an unconstrained formulation:

$$F(\mathbf{x}) = h_1(\mathbf{x}) + \frac{10^6}{2} \sum_{i=1}^{6} w_i \cdot g_i(\mathbf{x})^2$$

Applying AMGPRA

$\rightarrow$ Redefine the optimization problem as a **reliability problem**: we seek the probability that the system's cost exceeds a specific threshold τ:

$$g(\mathbf{x}) = \tau - F(\mathbf{x})$$

Goal:

Using the provided optimum $\mathbf{x}_{opt}$, investigate the probability of being 10% to 30% above the optimal performance

From validation to application

1st phase -Validation (achieved)

- Successful Python implementation of **AMGPRA**, **mfEGRA**, and **AK-MCS**
- **Reproducibility:** Validation by reproducing results from the original papers with close accuracy

2nd phase: Transition to solar

- Developing a Python-based *wrapper* to interface our algorithms with the C++ *Solar* simulator
- **Bi-fidelity setup:** Configuring the environment to test a dual-fidelity approach
- **Objective:** Quantify **reliability** of $F(\mathbf{x})$ by estimating the probability of exceeding the optimal threshold by 10% to 30%

Conclusion

Conclusion

Summary

- Multi-fidelity GP metamodel reduces computational cost
- **AMGPRA**: selects both the next point *and* the fidelity level, and the collective learning function (CLF) evaluates the global impact of a new point while improving efficiency vs. mfEGRA
- **Results**: Comparable or better accuracy with lower computational cost
- **Limits:** scalability with respect to the input dimension, lack of a publicly available implementation, parameters fixed empirically

Thank you !

References

- O. Ditlevsen, and HO. Madsen, Structural reliability methods, John Wiley Sons, 1996
- B. J. Bichon, M. . Eldred, L. P. wiler, . Mahadevan, and J. M. McFarland, "Efficient global reliability analysis for nonlinear implicit performance functions," AIAA J., vol. 46, no. 10, pp. 2459–2468, 2008.
- B. Echard, N. Gayton, and M. Lemaire, AK-MCS: An active learning reliability method combining Kriging and Monte Carlo Simulation, in "Structural Safety", vol.33, no.2, 2011.
- A. Chaudhuri, A. N. Marques, and K. E. Willcox, mfEGRA: Multifidelity Efficient Global Reliability Analysis, 2020.
- C. Zhang, C. Song, and A. Shafieezadeh, Adaptive Reliability Analysis for Multi-fidelity Models using a Collective Learning Strategy, in

AMGPRA as an algorithm I

Algorithm 1:

Input: Initial DoE $\mathcal{X}_0$, Monte Carlo sample size N_{MCS}
Output: Estimated probability of failure $\hat{P}_f$

```
procedure AMGPRA(X_0):
    X ← X_0, Generate S, Train multi-fidelity GP ĝ on X
    while True do
        Compute P̂_f, Err_max and COV_P̂f on S using ĝ
        if Err_max ≥ 0.001 then
            Select (x_new, l_new) via the CLF
            if l_new = 0 then
                Add x_new to all fidelities in X
            Retrain ĝ on updated X
        else if COV_P̂f > 0.05 then
            Generate ΔS and update S ← S ∪ ΔS
        else
            return P̂_f
```